ATTACK DETECTION IN SOCIAL PLATFORMS USING FUZZY AND INCREMENTAL NEURAL NETWORK

PHASE I REPORT

Submitted By

HARINISHREE K 3122225001035

MUTHUSELVI K 3122225001074

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Department of Computer Science and Engineering

Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University)
Kalavakkam - 603110

October 2026

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled **"ATTACK DETECTION IN SOCIAL PLATFORMS USING FUZZY AND INCREMENTAL NEURAL NETWORKS"** is the *bonafide* work of **HARINISHREE. K (312225001035)**, **MUTHUSELVI. K (312225001074)** who carried out the project work under my supervision.

Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. T.T. MIRNALINEE
HEAD OF THE DEPARTMENT
Professor
Department of CSE
SSN College of Engineering
Kalavakkam - 603 110

Dr. SURESH JAGANATHAN
SUPERVISOR
Associate Professor
Department of CSE
SSN College of Engineering
Kalavakkam - 603 110

Submitted for project viva-voce examination held on. . . . . .

EXTERNAL EXAMINER **INTERNAL EXAMINER**

ACKNOWLEDGEMENTS

We thank GOD, the almighty for giving our strength and knowledge to do this project.

We would like to thank and deep sense of gratitude to my guide **Dr.SURESH JAGANATHAN**, Associate Professor, Department of Computer Science and Engineering, for his valuable advice and suggestions as well as his continued guidance, patience and support that helped our to shape and refine our work.

Our sincere thanks to **Dr. T.T. MIRNALINEE**, Professor and Head of the Department of Computer Science and Engineering, for her words of advice and encouragement. We would like to thank **Dr.B.Bharathi** project Coordinator, our Project panel members for their valuable suggestions throughout this project.

We also express our appreciation to **Dr. S. RADHA**, Principal, for all the help she has rendered during this course of study.

We would like to extend our sincere thanks to all the teaching and non-teaching staffs of our department who have contributed directly and indirectly during the course of my project work. Finally, We would like to thank our parents and friends for their patience, cooperation and moral support throughout our life.

HARINISHREE K **MUTHUSELVI K**

ABSTRACT

Social Platform attacks like Phishing, baiting, pretexting, whaling, and malicious URL spreading attacks are challenging due to unstructured text, diverse URL patterns, and ever-changing attack patterns. This paper presents a memory-efficient incremental learning approach for social engineering attack detection using email text and URL feature data from dynamically streaming data.The proposed system uses a dual-pipeline feature extraction architecture. The first pipeline uses TF-IDF and Truncated SVD with a Stacked Autoencoder (SAE) to extract 100-dimensional latent features that represent semantic and structural patterns. The second pipeline adds a Fuzzy Probability-Guided Feature Descriptor Generator (Fuzzy-PGFDG) that extracts five fuzzy membership descriptors: Typicality, Density, Homogeneity, Outlier Score, and Fuzzy Entropy, to represent uncertainty and class membership. The results are combined into a single 124-dimensional hybrid feature vector.To handle concept drift and catastrophic forgetting, a Data Augmented Incremental Learning (DAIL) approach is used to synthesize new data from previous batches using a column-alternating interleaving scheme, without requiring storage of raw historical data. An Extreme Learning Machine (ELM) classifier is incrementally updated with dynamically scaled hidden neurons to ensure scalability and efficiency.

TABLE OF CONTENTS

ABSTRACT **iii**

LIST OF TABLES **vii**

LIST OF FIGURES **viii**

1 INTRODUCTION **1**

1.1 MOTIVATION 3

1.2 BACKGROUND 5

1.3 PROBLEM DEFINITION 7

1.4 ORGANIZATION OF THE REPORT 9

2 LITERATURE SURVEY **11**

2.1 PAPERS RELATED TO THE PROBLEM IDENTIFIED 11

2.1.1 Fuzzy Deep Neural Stacked Autoencoder (FDN-SA) . . . 11

2.1.2 LSTM-CNN Hybrid Deep Learning Architecture 12

2.1.3 SEADer++ v2 12

2.1.4 Data Augmented Incremental Learning (DAIL) 13

2.1.5 Augmented 1D-CNN with Recurrent Layers 13

2.1.6 Variational Autoencoders for URL Analysis 14

2.1.7 Deep Learning for Cyber Threat Mitigation 14

2.2 RESEARCH GAPS 15

2.3 RESEARCH OBJECTIVES 16

3 PROPOSED METHODOLOGY **19**

3.1 ARCHITECTURE DESIGN 19

3.2 ELM Based Classification Architecture 21

3.3 MODULES 23

3.3.1 Data Loading 25

3.3.2 Preprocessing Module 26

3.3.3 Stacked Autoencoder (SAE) Module 30

3.3.4 Combined Feature Module 33

3.3.5 Fuzzy Feature Generation 34

3.3.6 Data Augmentation 38

3.3.7 ELM Classifier 42

3.3.8 Incremental Learning 44

4 EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS **47**

4.1 DATASET DESCRIPTION 47

4.1.1 Dataset Composition 47

4.1.2 Dataset Distribution 48

4.1.3 Feature Representation 49

4.2 ECOSYSTEM 50

4.2.1 Hardware Configuration 50

4.2.2 Software Environment 50

4.3 EXPERIMENTS CONDUCTED 51

4.3.1 Incremental Learning Scenario Design 51

4.3.2 Stacked Autoencoder Experiment 53

4.3.3 Fuzzy-PGDFG Experiment 54

4.3.4 Classifier Training Experiment - ELM 57

4.4 PERFORMANCE ANALYSIS 57
4.4.1 SAE - ELM Analysis 57
4.4.2 Original Features+Fuzzy Features - ELM Analysis 58
4.4.3 Day-wise Incremental Performance - ELM 59
4.4.4 ELM Hyperparameter Selection Analysis 59
4.4.5 Complete Tabulation of Results 61

5 SOCIAL IMPACT AND SUSTAINABILITY 63
5.1 SOCIAL IMPACT 63
5.2 SUSTAINABILITY 64

6 CONCLUSION AND FUTURE WORK 65

REFERENCES 65

LIST OF TABLES

2.1 Identified Research Gaps and Proposed Research Objectives . . . 17

4.1 Dataset Description 47
4.2 Dataset Distribution 49
4.3 Day wise data configuration 52
4.4 Per-class augmentation outcomes 56
4.5 Performance Comparison of ELM Configurations 1 58
4.6 Performance Comparison of ELM Configurations 2(Hybrid Model) 59
4.7 Day-wise Incremental Analysis 60
4.8 Gap-Penalized Hyperparameter Selection Across Incremental Days 60
4.9 Day-wise Training Distribution and Incremental Performance . . . 61

LIST OF FIGURES

3.1 Architecture Design of Proposed System 20
3.2 PGFDG-Based Feature Extraction 35
3.3 Data Augmentation- Column Alternating Approach 39

4.1 Test Accuracy Comparison Across Four Incremental Learning Days 61

CHAPTER 1

INTRODUCTION

Social media platforms have been identified as the main channel of digital communication, but because of their open and unstructured nature, they are highly vulnerable to social engineering attacks like phishing, baiting, pretexting, whaling, and malicious URL attacks. These types of attacks use linguistic tricks, deceptive URLs, and behavioral patterns to target the security of users. The reason why it is difficult to detect these attacks is that the discriminative information is distributed across multiple heterogeneous channels like text semantics, URL graph properties. Moreover, the attack patterns keep on varying, leading to concept drift and catastrophic forgetting in the conventional static learning models.

To overcome these issues, the proposed system brings forth a multi-phase detection framework with a structured architecture based on the text and URL information of the emails. The preprocessing phase of the system is responsible for the normalization, lemmatization, URL segmentation, and statistical standardization of the feature representation. The feature extraction phase is carried out using a two-pipeline approach. The first pipeline consists of TF-IDF vectorization and Truncated SVD, followed by the application of a Stacked Autoencoder (SAE) for the extraction of a 100-dimensional deep latent feature representation through unsupervised reconstruction learning.

The second pipeline provides a Fuzzy Probability-Guided Feature Descriptor Generator (Fuzzy-PGFDG) to capture the uncertainty and boundary effects that

are typical of social engineering data. Based on the manually designed linguistic and URL feature descriptors, five fuzzy membership values are calculated: Typicality, Density, Homogeneity, Outlier Score, and Fuzzy Entropy, which capture the degree of belongingness to the classes in a probabilistic environment. The final feature vectors from the deep SAE-based pipeline and the fuzzy descriptor pipeline are combined to create a hybrid feature vector of dimension 124, which allows for complementary learning in the structured, statistical, and uncertainty domains.

To facilitate adaptability in the presence of dynamically changing data streams, the framework incorporates a Data Augmented Incremental Learning (DAIL) paradigm that retains knowledge of previous classes using representative samples instead of retaining the raw data. This approach helps to alleviate the problem of catastrophic forgetting while ensuring memory efficiency and scalability. Classification is carried out by an Extreme Learning Machine (ELM) with hyperparameter tuning for different hidden neuron settings and activation functions. Model selection is driven by a gap-penalized criterion that balances empirical risk minimization and generalization strength. The ELM is trained incrementally by replacing the existing models. The combination of deep representation learning, fuzzy probabilistic modeling, incremental learning, and optimized classification provides a scalable and robust framework for dynamic social engineering attack detection.

1.1 MOTIVATION

The growing frequency of social media attacks on social media platforms has created an urgent demand for clever, automated detection solutions. Social engineering attacks exploit users' trust by using malicious URLs, false email subject lines, and manipulative message content. These assaults are far more difficult to detect since they target users' human psychology rather than the system's technical characteristics. This is why automated and adaptive detection is a key need in today's cyber security.

The dynamic and unstructured nature of social media and email messages is one of the primary motivations for pursuing this topic. An attack can include multiple signs at the same time, including password-stealing phrases in everyday communication, urgent messages to press the user, and an attack URL embedded in the message. A system that can simultaneously assess the message content and URL structure information is necessary for identifying such assaults.

The existing machine learning solutions have been constrained by their reliance on manually designed features, which are time-consuming to design and do not effectively represent the deeply hidden and nonlinear patterns that exist in complex email and URL data. This drawback led to the incorporation of a Stacked Autoencoder(SAE) to automatically discover meaningful and compact representations from the raw input data through unsupervised reconstruction, without the need for manual feature selection.

Although the deep features obtained from the autoencoder are capable of representing the structural and statistical patterns, they do not represent the uncertainty or class membership of the ambiguous samples that are located near

the decision boundaries. This drawback led to the incorporation of a Fuzzy Probability-Guided Feature Descriptor Generator (Fuzzy-PGFDG) to enhance the feature representation with meaningful fuzzy membership values such as Typicality, Density, Homogeneity, Outlier Score, and Fuzzy Entropy.

The evolution of attack patterns over time is another, and no less significant, explanation. Attacks using social engineering are constantly evolving, with new tactics and types of attacks always emerging. The ability of an attack detection model to identify new, undiscovered attack types will soon be rendered obsolete if it was trained and fixed on a static dataset. Retraining the model from scratch each time fresh data is received is computationally impractical and renders all previously acquired knowledge unusable. To learn incrementally, DAIL (Data Augmented Incremental Learning) was used to update the model with new attack data. Additionally, a column-alternating augmentation technique was used to retain knowledge of previously seen classes by creating artificial data samples that represented attack data from previous classes without storing it.

Finally, the reason for choosing ELM as the classifier was its efficiency in computational calculation, analytical calculation of weights, and its outstanding generalization performance when dealing with high-quality hybrid features. The ability to be applied in situations where rapid updates of the model are needed with every new batch of data makes it a perfect fit for the proposed system's application.

1.2 BACKGROUND

With the growing popularity of social media and online communication platforms, the trend of social platform attacks has intensified. These platforms are used by social engineers to attack a large number of users at once by sending misleading text messages, inserting malicious URLs, and impersonating trusted sources. The issue of detecting harmful actions has remained difficult due to the unstructured, diversified, and continuously changing nature of social media information. Conventional detection techniques, such as blacklisting, rule-based filtering, and keyword matching systems, are inadequate as attackers often change their behavior patterns frequently, making it easy for them to evade detection by changing the wording, URL format, or source address.

To make detection more accurate and robust, there has been a gradual movement in the research community toward the use of automated computational models that can process multiple data sources together. Early machine learning models relied heavily on manual feature engineering, where experts in the domain designed features manually. This is a time-consuming, domain-specific, and inadequate approach to feature engineering, as it is not capable of extracting the underlying, deeply hidden non-linear patterns that exist in the URL structure and natural language text content.

To address this drawback, deep learning-based feature extraction has been employed extensively. Among these, the Stacked Autoencoder (SAE) has been shown to be very successful. A Stacked Autoencoder is a dimensionality reduction technique that reduces high-dimensional input data into a compact, lower-dimensional encoded form by learning to reconstruct the input data using a

bottleneck architecture of stacked encoder and decoder layers. For this project, individual autoencoders are trained for text features (reducing a 200-dimensional TF-IDF SVD feature to 50 dimensions) and URL features (reducing a 68-dimensional feature to 50 dimensions), resulting in a combined 100-dimensional deep feature vector. Importantly, the autoencoder is trained only on Day 1 data and its weights are locked for all subsequent incremental days, providing a stable feature space for the entire learning process.

Though autoencoders perform admirably on learning structural and distributional patterns, they are deterministic in nature and fail to capture the gradual and uncertain nature of class boundaries, which is a major drawback in distinguishing semantically similar attack categories like phishing and pretexting attacks. To overcome this, the Fuzzy-PGFDG pipeline has been introduced, which uses fuzzy logic-based feature generation. The concept of fuzzy sets, first proposed by Zadeh, allows the representation of partial membership in classes, as opposed to binary membership.Based on membership functions that are defined by the estimates of Mahalanobis distance, neighborhood density, and entropy, five fuzzy features: typicality, density, homogeneity, outlier score, and fuzzy entropy are derived for each sample in this setup. Once these features are incorporated into the manually designed 19-dimensional feature space, a 24-dimensional fuzzy-hybrid feature vector is formed. This feature vector is then concatenated with the output of the autoencoder to form the final 124-dimensional feature vector.

The assumption of static data distributions, in which the models are trained just once and are not modified, is one of the most important missing assumptions in the detection systems in use today. In actuality, new attack patterns are created

throughout time, and social engineering assault patterns are always changing. This necessitates the use of an incremental 7 learning strategy, which allows the detection model to be updated with each new batch of data while retaining the knowledge gained from earlier batches.In order to solve this problem, Madhusudhanan and Jaganathan's DAIL (Data Augmented Incremental Learning) architecture uses column-alternating data augmentation, a method that creates new data by switching the values of the feature columns of pairs of pre-existing samples. This prevents catastrophic forgetting while being memory-efficient since it guarantees that the new samples are always in the same feature space as their parent cluster, enabling the model to retain its prior class knowledge without having to store the actual data.

As an incremental learning classifier for classification, the Extreme Learning Machine (ELM) is used. Unlike conventional backpropagation-based networks, ELM is a single-hidden-layer feedforward network that trains much more quickly since the input weights are randomly allocated and the output weights are calculated analytically using the Moore-Penrose pseudoinverse. In this project, an automated hyperparameter search is used to test different hidden layer sizes and activation functions. The best combination is chosen based on a gap-penalized scoring metric that prioritizes test accuracy over large train-test splits across all four days of incremental learning.

1.3 PROBLEM DEFINITION

Massive volumes of unstructured and diverse data are generated every second by social media platforms, which makes them an ideal target for malicious URL

distribution, phishing, baiting, pretexting, and whaling attacks. Conventional rule- and signature-based security systems struggle to identify these kinds of assaults since they rely on human trust and behavior patterns. A learning model based on historical data becomes obsolete very quickly since the patterns of these attacks are not static; new patterns and attack types are constantly emerging.

The data's high dimensionality and multimodality present the first technical difficulty. Two distinct modalities—the language and structure of the URL and the email text—are used to characterize the attack elements. Preprocessing pipelines and feature extraction techniques vary per modality. The modalities' unprocessed features are redundant, noisy, and high-dimensional. This necessitates the use of efficient representation learning methods capable of converting raw features into a low-dimensional representation that is noise-free and retains discriminative information.

Due to the combined need to adjust to changing attack patterns, different kinds of data, and the shortcomings of traditional models, a system must be able to execute incremental learning in addition to deep feature learning and fuzzy-based feature extraction. All three needs cannot be met at the same time by the current methods since they rely on feature set knowledge, ignore the ambiguity of class boundaries, or necessitate retraining the entire system if new attack classes emerge. Through the integration of a DAIL-based incremental learning framework with column-alternating augmentation, a Fuzzy-PGFDG for interpretable membership function modeling, and a Stacked Autoencoder for deep feature learning, the suggested system simultaneously satisfies all three objectives.

1.4 ORGANIZATION OF THE REPORT

This project report is divided into chapters, with each chapter describing a significant component of the work performed for the detection of social engineering attacks through multi-modal data processing, feature extraction, and neural network classification. The project report starts with Chapter 1, which describes the introduction to the project by explaining the motivation, introducing the background of the study, describing the problem statement, and finally explaining the contents of the report. Chapter 2 of the report discusses the literature survey, which includes an explanation of the existing literature on social engineering attack detection, multi-modal data analysis, data preprocessing techniques, autoencoder-based feature extraction,fuzzy feature extraction and neural network classifiers.

Chapter 3 explains the proposed methodology in detail. It discusses the system architecture, the modules, and the preprocessing tasks involved in URL, text data. The chapter also deals with the operation of the stacked autoencoder and the classification process using ELM. Pseudocode and flowcharts are used to explain the implementation process in detail. Chapter 4 deals with the experimental setup. It explains the dataset, the software and hardware environment, and the experiments performed. The results are also discussed, followed by performance analysis, observations, and key inferences.

Chapter 5 discusses the social effects and sustainability of the proposed system, focusing on its role in ensuring safe online communication and developing a long-term awareness of cybersecurity. Chapter 6 presents the conclusion of this research effort and suggests possible future improvements, such as better model

accuracy, real-time analysis, and the ability to handle larger datasets or other types of attacks. Finally, Chapter 7 presents a list of all the references cited in this report, following the proper format for academic citation. These chapters, therefore, provide a comprehensive and organized way of presenting the research process, from problem identification to the development of the system and analysis of its results.

CHAPTER 2

LITERATURE SURVEY

2.1 PAPERS RELATED TO THE PROBLEM IDENTIFIED

2.1.1 Fuzzy Deep Neural Stacked Autoencoder (FDN-SA)

The FDN-SA system, developed by Vidyasri and Suresh, combines fuzzy logic and deep learning to improve phishing detection in social engineering scenarios. The approach reduces dimensionality without sacrificing discriminative features by using a stacked autoencoder to learn layered, abstract representations from complicated datasets made up of URLs, email headers, and webpage elements. By addressing the vague and unpredictable character of attack signatures, fuzzy logic integration gives the classifier the ability to more robustly manage changing trends in threat behavior. In detecting sophisticated phishing attacks, empirical evaluation showed that FDN-SA performed better than traditional machine learning models, such as SVMs and Random Forests, by obtaining higher accuracy and lower false-positive rates. The method of combining deep autoencoding with fuzzy inference for sophisticated attack detection is validated by this model's capacity to capture both syntactic and semantic information.

2.1.2 LSTM-CNN Hybrid Deep Learning Architecture

For phishing detection, Ariyadasa et al. suggested a composite neural model that makes use of spatial and sequential analysis. By combining one-dimensional convolutional layers (CNNs) with long short-term memory (LSTM) networks, the design allows the framework to learn spatial properties in HTML content and temporal dependencies in URL character streams at the same time. This modular combination enables the system to detect malicious scripts contained in page structures as well as subtle modifications in web links. The system achieves great accuracy, outperforming earlier hybrid URL-HTML models, thanks to the separated training and subsequent merging of outputs via activation layers. Its applicability for real-time security applications is highlighted by its minimal preprocessing and lack of external feature engineering, as well as its increased robustness and generalization for threats that are not yet visible.

2.1.3 SEADer++ v2

Lansley, Kapetanakis, and Polatidis: SEADer++ v2 uses machine learning and natural language processing to detect social engineering techniques in text messages. In order to identify linguistic elements indicative of malevolent activity, such as urgency cues, grammatical abnormalities, and manipulative language patterns, the system preprocesses chat and email data using tokenization, stop-word removal, stemming, and syntactic analysis. Both overt and covert signs of social engineering are captured by the seven-feature representation, which is then input into a variety of classifiers, such as Random Forest, feedforward neural networks, and support vector machines. Viability for

organizational security deployment is established by testing on semi-synthetic datasets, which show notable gains in detection accuracy and false-negative reduction.

2.1.4 Data Augmented Incremental Learning (DAIL)

DAIL, a completely unsupervised method for preserving model effectiveness in dynamic attack situations, was presented by Madhusudhanan and Jaganathan. By using augmentation to create synthetic datasets, the method gradually updates pre-existing models without the need for extensive retraining cycles or human labeling. While maintaining low computational costs, this approach significantly increases the classifiers' adaptability and scalability in the face of quickly changing social engineering patterns. Performance investigation shows that DAIL performs well in clustering and classification on continuously changing data distributions, which makes it especially useful in situations where attack vectors are constantly changing and labeled data is hard to come by.

2.1.5 Augmented 1D-CNN with Recurrent Layers

For phishing email detection, Altwaijry et al. suggested augmented one-dimensional CNN models (1D-CNNPD) combined with sophisticated recurrent architectures including LSTM, Bi-LSTM, GRU, and Bi-GRU. While bidirectional recurrent processing gathers contextual information from both forward and backward directions, allowing the model to acquire intricate attack signatures, one-dimensional convolution examines sequential text patterns. By

achieving accuracy on benchmark datasets, our multi-layered recurrent technique shows how well it works to combine several neural paradigms for improved phishing detection.

2.1.6 Variational Autoencoders for URL Analysis

Prabakaran et al. presented improved deep learning techniques for robust reconstruction and unsupervised URL feature learning using variational autoencoders (VAE). Through reconstruction error anomaly detection, VAE's probabilistic architecture makes it possible to develop compact, generative representations of URL patterns, which makes it easier to identify phishing variants that haven't been seen before. The method confirms VAE's better capacity to handle high-dimensional, complicated URL datasets in comparison to deterministic autoencoders, with 98.2% accuracy and proven robustness to zero-day phishing patterns.

2.1.7 Deep Learning for Cyber Threat Mitigation

Tyagadurgam et al. demonstrated cutting-edge deep learning methods created especially for mitigating cyberthreats and preventing phishing attacks. In order to achieve high-performance threat detection in production contexts, the research discusses cutting-edge neural architectures, training techniques, and deployment tactics.

2.2 RESEARCH GAPS

Limited Study on Combining Multiple Types of Data

The majority of current systems exclusively analyze one kind of data, like text, URLs, or metadata. The procedure is typically straightforward and poorly assessed, even when several types are integrated. Very little study has been done on how sophisticated feature extraction strategies can enhance multi-class social engineering detection or which sort of data contributes most to detection accuracy.

Lack of Comparison Between Heavy Deep Models and Faster Learning Models

As stated in 2.1, complicated deep learning models are the main focus of current research, whilst quicker and simpler model like Extreme Learning Machines are seldom In order to determine the trade-off between accuracy, training time, and computational efficiency for real-world deployment, the several approaches discussed above are examined.

Limited Focus on Multi-Class Data

The majority of current research focuses on binary phishing/legitimate classification or, at most, ternary scenarios, ignoring the complex multi-class nature of real social engineering threats.The literature lacks a thorough assessment of detection systems' resilience across various organizational contexts, unequal real-world class distributions, and fine-grained attack categorizations (phishing, whaling, pretexting, baiting, legitimate), as summed up

in 2.1. Furthermore, the majority of assessments rely on controlled, openly accessible datasets that might not accurately represent linguistic variances, industry-specific attack vectors, or organizational-specific communication patterns.

Insufficient Attention to Real-Time Deployment and Explainability

Practical deployment aspects including inference speed, computing overhead, and scalability for large-volume social platform data have received relatively little attention in the majority of previous studies, which primarily concentrate on accuracy measures.In addition, the majority of deep learning models function as black boxes with little support for explainability, as summed up in 2.1.Consequently, it is challenging for security analysts to comprehend and justify classification results. Furthermore, there is a gap in our knowledge of security because there is insufficient research on whether multi-modal systems provide greater resilience against hostile manipulation or evasion strategies.

2.3 RESEARCH OBJECTIVES

The primary goal of this study is to create a comprehensive multi-modal detection framework that combines information from multiple sources to more accurately and reliably identify and categorize social media attacks.

- Develop two separate specialized neural models - autoencoders for each data The types are text, URL.
- Extract the most relevant patterns from each data source independently.

- Combine features from all two sources into a unified representation.

TABLE 2.1: Identified Research Gaps and Proposed Research Objectives

Category	**Identified Gaps**	**Proposed Direction & Research Objectives**
Multi-Modal Integration & Feature Fusion	Limited multi-modal integration; insufficient evaluation of modality-specific feature extraction effectiveness.	**1.** Design stacked autoencoders for each modality (text, URL). **2.** Investigate feature-level fusion by concatinating modality specific stacked AutoEncoder and analyze the performance. **3.** Validate improvements in accuracy and generalization over single-modality approaches.
Multi-Class Granularity & Generalization	Binary/ternary dominance; limited multi-class evaluation; insufficient cross-domain robustness assessment; class imbalance challenges.	**1.** Develop five-class classifier (phishing, whaling, pretexting, baiting, legitimate). **2.** Use balanced dataset (500 per class). **3.** Identify modality contributions for distinguishing rare vs. common attacks.
Real-Time Deployment Feasibility & Explainability	Underreported efficiency metrics; lack of deployment feasibility analysis; limited explainability; no analysis of modality redundancy.	**1.** Report training time, latency, memory, throughput for ELM. **2.** Implement modality-specific feature attribution. **3.** Evaluate robustness when modalities are partially missing or degraded.

The second aim is to provide practical guidance that can be used in the real world deployment, by doing a profound comparative analysis between two different Neural architectures for security practitioners. Implement and test ELM, which represents deep supervised learning.

- Implement and test Extreme Learning Machine (ELM) representing fast analytical learning.
- Measure and report training time, inference latency, memory usage, and detection accuracy
- Clearly indicate to practitioners which architecture applies or choose based on their needs.
- Help organizations select classifiers based on available computational resources and performance requirements.

The tertiary objective the extends detection capability beyond binary classification to achieve fine-grained attack categorization and system transparency improvement and resiliance.

- Develop a five-class detection system distinguishing phishing, whaling, Pretexting, baiting, and legitimate datas.
- Measure performance metrics separately for each attack type to identify system strengths and weaknesses.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 ARCHITECTURE DESIGN

The system architecture is developed in such a way that it can handle the incoming data in various batches and update the feature representation dynamically for efficient social engineering attack detection. The system starts with a series of data batches, in which every batch goes through a Feature Updation phase. This module is tasked with the responsibility of preparing and optimizing the input data by using various preprocessing techniques such as cleaning, normalization, transformation, and extraction of key attributes. As the system receives the data in batches and not a static dataset, the Feature Updation module is used to process every batch independently and transform it into a new feature set as illustrated in Fig 3.1

Once the feature updation process is completed, the updated features are passed to the Neural Network Module. The Neural Network Module is the central part of the architecture and is tasked with learning the discriminative patterns from the updated features and developing models for classification. As evident from the architecture, each batch is used to develop a model that is referred to as M_1, M_2, M_3, and so on.

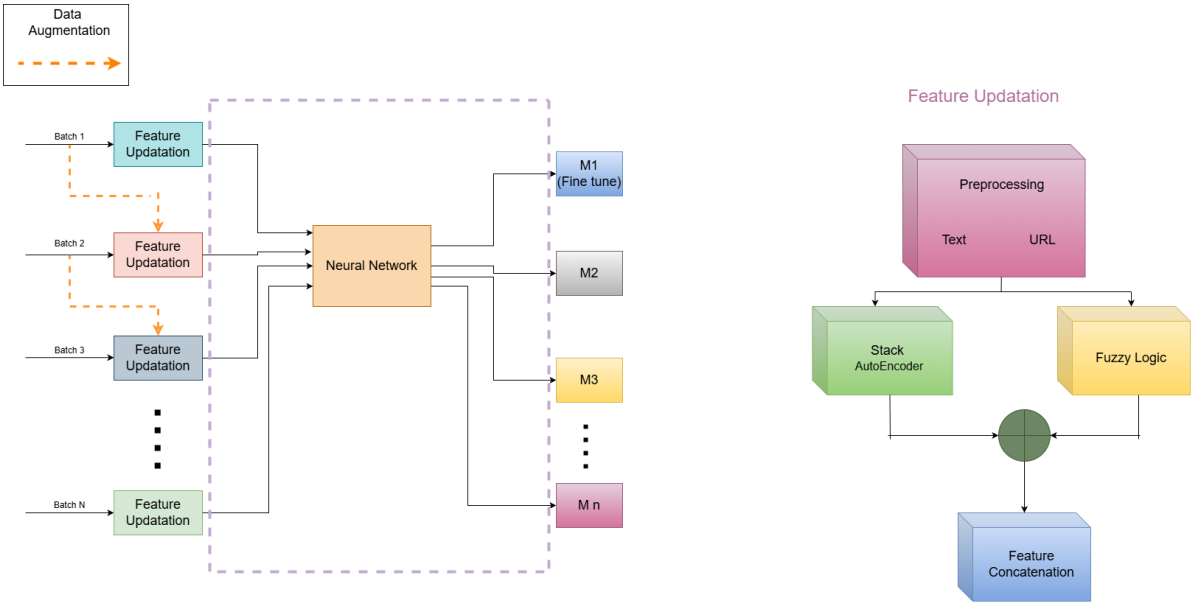


FIGURE 3.1: Architecture Design of Proposed System

In parallel to the process of updating features in batches, there is also a Data Augmentation module in the system that introduces diversity and variability to the training data. Whether it is the creation of new data samples synthetically or the creation of new samples from the existing ones, the data augmentation module is responsible for enhancing the generalization capability of the neural network as well as ensuring that it does not overfit. The process of updating features in batches and data augmentation ensures that the learning process remains stable even when the input batches have different distributions and structures.

On the right side of the architecture, the internal workflow of the Feature Updation Module has been shown in more detail. The Preprocessing module processes two types of data, Text and URL, thereby ensuring that both types of data are properly preprocessed. The preprocessed data is then fed to a Stacked Autoencoder, which reduces the dimensionality of the data and extracts meaningful latent features. Along with the autoencoder, a Fuzzy Logic module is

used to process the uncertainties and linguistic nuances that are present in the data. The output of both the autoencoder and the fuzzy logic module is then combined using a Feature Concatenation module, resulting in a homogeneous feature vector that is then fed to the neural network for training and model development.

Overall, the architecture is designed to enable continuous learning, multi-modal feature refinement, and incremental model synthesis. By integrating preprocessing, fuzzy logic processing, stacked autoencoder-based feature extraction, and batch-based neural learning, the architecture provides a well-organized and scalable data pipeline from raw data to the final attack prediction.

3.2 ELM Based Classification Architecture

The architecture of the system is designed to detect social engineering attacks by analyzing two different types of input data, which are text and URL features, by combining them into a single feature space. The system begins with the Data Loading module, where the data set containing the two types of input data is loaded into the system. Once the data is loaded, it is divided into its respective components and then sent to the two preprocessing paths.

The Text Preprocessing module performs operations such as text cleaning, lemmatization, removal of noise, and transformation into numerical vector form using TF-IDF vectorization followed by Truncated SVD for dimensionality reduction. The URL Preprocessing module extracts structural and lexical

characteristics from the raw URL strings — including URL length, dot count, slash count, HTTPS presence, and character-level n-gram TF-IDF features — normalizes the values, and prepares them for feature learning.

After preprocessing, each data modality is sent to its corresponding Stacked Autoencoder (SAE) component — SAE Text and SAE URL. These autoencoders reduce the dimensionality of the processed inputs and learn meaningful latent feature representations. Instead of manually selecting features, the stacked autoencoders automatically extract hidden patterns and compress the information while preserving the critical characteristics of each data type. The text autoencoder follows a $200 \rightarrow 100 \rightarrow 50 \rightarrow 100 \rightarrow 200$ architecture, producing a 50-dimensional encoded text representation, while the URL autoencoder follows a $68 \rightarrow 60 \rightarrow 50 \rightarrow 60 \rightarrow 68$ architecture, producing a 50-dimensional encoded URL representation. Both the autoencoders are trained only on the Day 1 data, and their weights are frozen for the rest of the days. This ensures that the feature space is stable throughout the learning process.

The encoded outputs are then combined in the Combined Feature module. The Combined Feature module utilizes feature-level combination by concatenating the two 50-dimensional latent outputs into a combined 100-dimensional feature vector from Pipeline 1. Simultaneously, the Fuzzy-PGFDG module from Pipeline 2 generates a second 24-dimensional feature vector that encompasses 19 manual text and URL features expanded with 5 fuzzy descriptors: Typicality, Density, Homogeneity, Outlier Score, and Fuzzy Entropy. The output from Pipeline 1 and Pipeline 2 is then concatenated to generate the final 124-dimensional combined feature vector that encompasses a broad spectrum of information from both deep autoencoder latent outputs and fuzzy interpretable features.

This combined 124-dimensional feature vector is then fed into the Extreme Learning Machine (ELM) classifier. The ELM is a single-hidden-layer feedforward neural network that trains much faster than backpropagation-based models. The weights and biases of the hidden layer of the ELM are randomly generated and then fixed, and the output weights are determined in a single step using the Moore-Penrose pseudoinverse.

Finally, the trained ELM predicts the attack type using the combined 124-dimensional feature input. The ELM classifies each input instance into one of five categories: Baiting, Legitimate, Phishing, Pretexting, or Whaling. The ELM model of the previous day is intentionally deleted after each update, and only the new ELM model is saved, which ensures that the memory usage remains bounded for all incremental days. In conclusion, the architecture provides a systematic and optimized workflow from data loading to attack prediction, which covers preprocessing, automatic deep feature extraction, fuzzy feature generation, feature fusion, and analytically derived incremental classification.

3.3 MODULES

The proposed system is designed using a modular approach, where each module is designed to serve a distinct and critical purpose in the entire process of social engineering attack detection. The proposed system deals with two different types of data: textual content and URL features. Therefore, the proposed system consists of distinct modules designed specifically to deal with each type of data in an effective manner. The process begins with the Data Loading module, which loads the dataset and organizes it in the required formats. This module ensures

that all the data is organized in the required and separated into text and URL components so that the subsequent stages receive clean and well-organized inputs.

Once the data is loaded, it is then passed through the Preprocessing module, which is tasked with the responsibility of preparing the data for further processing. Since each modality has its own unique features, two preprocessing pipelines are used. The text preprocessing entails the cleaning, tokenization, and vectorization of the raw text data, which makes it possible to transform the raw text data into numerical form. The URL preprocessing entails the extraction of structural and lexical features that help in the identification of malicious link patterns.

The processed data is then passed on to the Stacked Autoencoder (SAE) module. The SAE module is a very important part of the process of extracting significant hidden features from each type of data. Rather than using feature engineering, which is a time-consuming process, autoencoders are used to automatically extract hidden patterns and reduce the high-dimensional input into a lower-dimensional space. Autoencoders are employed independently for text data as well as URL modalities, ensuring that each type of data is mapped to its features based on its structural properties.

The output of the two autoencoders is merged in the Combined Feature module, where the combination of features at the feature level is performed. This module combines the latent features of all the modalities into a single feature representation. By combining the information from the text, URL data, the system obtains a better understanding of the samples, which helps in accurately

identifying the attack patterns. The combined feature vector is the final input to the classification models.

The Classification module consists of one neural network model: Extreme Learning Machine (ELM). That model use the fused feature vector to predict the attack type. ELM uses randomly generated hidden nodes to train quickly.Model can be used to distinguish benign data from different types of social engineering attacks.

Finally, the Output module relays the result of the prediction to the user by specifying the attack category that has been identified. The modular structure of the system from data loading to preprocessing, feature learning, feature fusion, classification, and finally to output ensures that the system is efficient and scalable enough to handle multi-modal inputs for attack detection.

3.3.1 Data Loading

The Data Loading process is the backbone of the entire detection system because it is responsible for the importation, organization, and proper structuring of all the data used in the entire process. In this project, the data consists of two forms of information: text information and URL-related information. Because both forms of information have different properties, the Data Loading module is responsible for ensuring that both forms of information are properly identified and separated for the next modules to handle them in the correct manner.

When the data set is received, it starts with the process of reading the input from its source, such as CSV files, and validating the integrity of the data. It makes sure

that there are no missing fields, that the data types are not incorrect, and that the formatting is consistent. The process also validates the number of samples, the alignment of the labels with the features, and breaks down the content into two structured parts. The text fields are stored in a separate structure so that they can be further preprocessed linguistically. The URL strings are stored in a separate area, awaiting URL-specific type extraction and cleaning.

Another significant task related to this step is the handling of errors and anomalies that are present in the raw data. These can be in the form of blank entries, incorrect entries, noisy symbols, or broken lines. The Data Loading is tasked with the preliminary filtering of the data to remove or point out erroneous data. In some cases, it is also tasked with documenting any discrepancies for later review, as a way of promoting transparency and debugging.

By the time the data loading process is completed, the system will have a well-structured format of text and URL, data. This is an important aspect because it will ensure that all the preprocessing modules receive the data in the exact format that they require.The Data Loading process is important because it will ensure the stability, reliability, and correctness of all the input data.

3.3.2 Preprocessing Module

The Preprocessing Module is a crucial component of the proposed system, as it attempts to transform the raw and unstructured inputs into clean, meaningful, and machine-readable formats. Considering the fact that the dataset comprises two distinct types of data: text and URL attributes, the preprocessing module comprises two distinct preprocessing pipelines. The two preprocessing pipelines

are designed in such a manner that they are capable of handling the nature of their respective data modalities. The preprocessing module ensures that noise is eliminated, inconsistencies are resolved, and all data formats are transformed into structured formats that are ready for processing and feature extraction. The preprocessing module will ultimately be responsible for preparing the data for the Stacked Autoencoder and neural classification models.

3.3.2.1 Text Preprocessing

The Text Preprocessing is concerned with the cleaning and normalization of the text data acquired from posts, messages, or descriptions available on social media platforms. The raw text data usually comprises unnecessary characters, emojis, URLs, repeated symbols, case variations, and unnecessary stopwords that do not add any value to the meaningful information. This module is responsible for the cleaning of the text data by eliminating all the unnecessary characters and normalizing the text. The process starts with the conversion of text to lowercase to ensure consistency, followed by the removal of special characters, punctuation, and unnecessary spaces. The tokenization process is then used to split the sentences into individual words or meaningful pieces. Stemming or lemmatization is used to convert words to their root form to eliminate redundancy in the vocabulary.

After the cleaning process is finished, the text data must be converted into a form of numbers that can be handled by machine learning algorithms. A technique commonly employed for this task is TF-IDF (Term Frequency-Inverse Document Frequency), which gives more significant to important words and reduces the effect

of frequently occurring words. The TF-IDF value of a word t in a document d is given by the following formula:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log\left(\frac{N}{\text{DF}(t)}\right) \tag{3.1}$$

where,

- $\text{TF}(t,d)$ is the frequency of term t in document d,
- N is the total number of documents, and
- $\text{DF}(t)$ is the number of documents containing term t.

This formula gives higher importance to terms that are frequent in one document but rare across others. In the context of attack detection, TF–IDF helps highlight contextual keywords and suspicious linguistic patterns often found in phishing, impersonation, or malicious messages.

However, TF–IDF vectors are often very high-dimensional, which can slow down the learning process and reduce model efficiency. To reduce dimensionality while preserving the essential meaning of the text,Singular Value Decomposition (SVD) is applied to the TF–IDF matrix. Mathematically, the decomposition is expressed as:

$$X_{\text{TF-IDF}} = U\Sigma V^T \tag{3.2}$$

where,

- U captures document-level patterns,
- Σ contains the most important "strengths" or weights of the patterns, and
- V^T represents word-level relationships.

That is, SVD helps in reducing the TF-IDF matrix, which is quite large, to its most significant components, so that the system can focus on the most significant linguistic aspects while ignoring the rest.

By carrying out the above-mentioned steps of cleaning, normalization, number conversion, and dimensionality reduction. The Text Preprocessing module ensures that only the most significant and compact representation of the text features are passed on to the feature extraction stage.

3.3.2.2 URL Preprocessing

The URL Preprocessing is intended to identify valuable structural and lexical patterns from the links of websites. URLs are known to hold important features of malicious activities, since attackers tend to incorporate deceptive components to deceive users. To identify these features, the preprocessing process tends to evaluate various components of each URL, including domain names, subdomains, path formats, query strings, and encoded characters. Features such as URL length, the number of numeric characters, the number of special characters, the presence of deceptive keywords, the use of HTTPS, or the presence of IP domain names are extracted and analyzed. Other lexical features might include character frequency patterns or entropy values.

After feature extraction, the values of these features are normalized to maintain numerical consistency for all samples. This is done to make the machine learning model consider all features equally. The normalization process is carried out using the Standard Scaler as follows:

$$X_{\text{norm}} = \frac{X - \mu}{\sigma} \tag{3.3}$$

where X is the original feature value, μ is the mean of the feature across all samples, and σ is the standard deviation. This transformation ensures that features with larger numerical ranges do not dominate those with smaller ranges, improving model performance and stability.

The resulting cleaned and structured URL features help the system distinguish between normal links and deceptive or malicious URLs, significantly contributing to accurate attack detection.

3.3.3 Stacked Autoencoder (SAE) Module

The Stacked Autoencoder (SAE) module is given the task of extracting hidden patterns from the preprocessed data and then reducing the dimensionality of the data while preserving the crucial information. Since the system handles two different types of inputs, namely text and URL, two different autoencoders are used: SAE Text and SAE URL. Both autoencoders are trained separately to extract the important hidden patterns based on their respective data types.

3.3.3.1 SAE Text Module

The purpose of the SAE Text module is to reveal the hidden patterns in the linguistic and semantic structures from the preprocessed text data. The text data is expected to have complex patterns, word relations, and hidden semantic information that are difficult to model using simple and manually designed features. The SAE Text module automatically discovers the patterns by compressing the input text using multiple layers, creating a latent space that captures the essential characteristics of the text.

In the encoding process, the input text is compressed step by step, and each layer learns to capture the essential relations among words, phrases, or context-dependent features. To introduce non-linearity and allow the network to model complex relations, LeakyReLU activation is used in the hidden layers. The LeakyReLU activation function is defined as:

$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{if } x \leq 0 \end{cases} \tag{3.4}$$

where $\alpha = 0.2$. This activation allows small negative values to pass through instead of setting them to zero, preventing neurons from becoming inactive and helping the network learn more robust representations.

The encoding process results in a compact vector representation that encodes the semantic essence of the text. The decoding process is tasked with the responsibility of recovering the original input from the latent vector, ensuring that the compressed vector is semantically meaningful. Although the reconstructed text is not used in

the system, this process ensures that the latent vector preserves the most important aspects of the text.

SAE Text reduces noise and redundancy by converting raw text into dense vectors, hence reducing the dimensionality of the input data. The representation form of the input data is of high quality and is an ideal input for the classifiers, which can easily classify the malicious content. SAE Text automates the feature extraction process of the input text data.

3.3.3.2 SAE URL Module

The SAE URL module is concerned with learning structural and behavioral patterns from URL data. URLs are known to have indicators of potential attacks, including suspicious subdomains, path patterns, query strings, and encoded characters. The SAE URL module is able to learn these patterns by compressing the URL features into latent vectors.

During the encoding process, the module gradually decreases the dimensionality of the URL features, learning the relationships between structural elements, character distributions, and other lexical features. The latent vector learned from this process captures the behavior of the URL, making it easier for classifiers to distinguish between malicious and benign behavior. During the decoding process, the input URL is reconstructed to ensure that the key information is captured in the compressed form.

The SAE URL module removes unnecessary information, focusing only on the information that is crucial in making a distinction between normal and malicious URLs. The SAE URL module enhances the performance of the MLP and ELM

classifiers by producing high-quality compact vectors. This module ensures that the system is able to process a large amount of URL information while still being accurate in attack detection.

3.3.4 Combined Feature Module

The Combined Feature module is a feature-level fusion module where the latent features extracted from the two Stacked Autoencoders are combined. Once each modality, text, and URL, has been represented as a compact feature vector, the vectors are combined to represent the input instance. This is an important step in the process as it enables the system to process both types of input simultaneously, thus enhancing its capability to identify complex attack patterns.

The concatenation of the latent vectors can be represented as:

$$X_{\text{fused}} = X^{\text{text}}_{\text{bottleneck}} \parallel X^{\text{url}}_{\text{bottleneck}} \parallel \tag{3.5}$$

where $X^{\text{text}}_{\text{bottleneck}}$and $X^{\text{url}}_{\text{bottleneck}}$are the latent vectors obtained from the SAE Text, SAE URL, and SAE Tabular modules, respectively. The symbol $\parallel$ denotes vector concatenation. This step simply concatenates the three vectors together, forming a single unified feature vector that aggregates information from all three modalities. In doing so, the classifier is provided with a more informative representation of each sample, one that aggregates individual and cross-modal patterns.

The module also maintains structural consistency of the aggregated vector for all samples by addressing feature alignment, dimension consistency, and order. This

structured unified representation serves as the input to the neural classifiers, allowing them to make more informed decisions. For instance, a phishing attack may contain suspicious text features, irregular URL features, and irregular metadata features; when all three modalities are aggregated, the system is able to identify these patterns more effectively.

The Combined Feature module consolidates multi-modal latent features into a single structured vector, enhancing the system's predictive power and supporting precise attack classification.

3.3.5 Fuzzy Feature Generation

To increase class discrimination and model uncertainty in the detection of social engineering attacks, a Probabilistic Gaussian Fuzzy Distribution Generator (PGFDG) is employed in the fuzzy feature extraction framework. The primary aim of this component is to convert the normalized manual features into fuzzy features that are uncertainty-aware.

As Shown in 3.2, firstly, 19 manual features (11 text features and 8 URL features) are extracted and normalized using StandardScaler to ensure stable statistical calculations. The normalized features are then processed class-wise to generate fuzzy features for each attack class.

For each class, statistical parameters such as mean, standard deviation, and median are computed. Based on these, different fuzzy attributes are derived. Typicality is derived using a Gaussian membership function to measure the degree of membership of the sample to the class. Core Membership and

Boundary Proximity measure the centrality and boundary samples for the cluster. Density Score measures the local density of the samples, while Centrality and Compactness measure the structural contribution of the class.

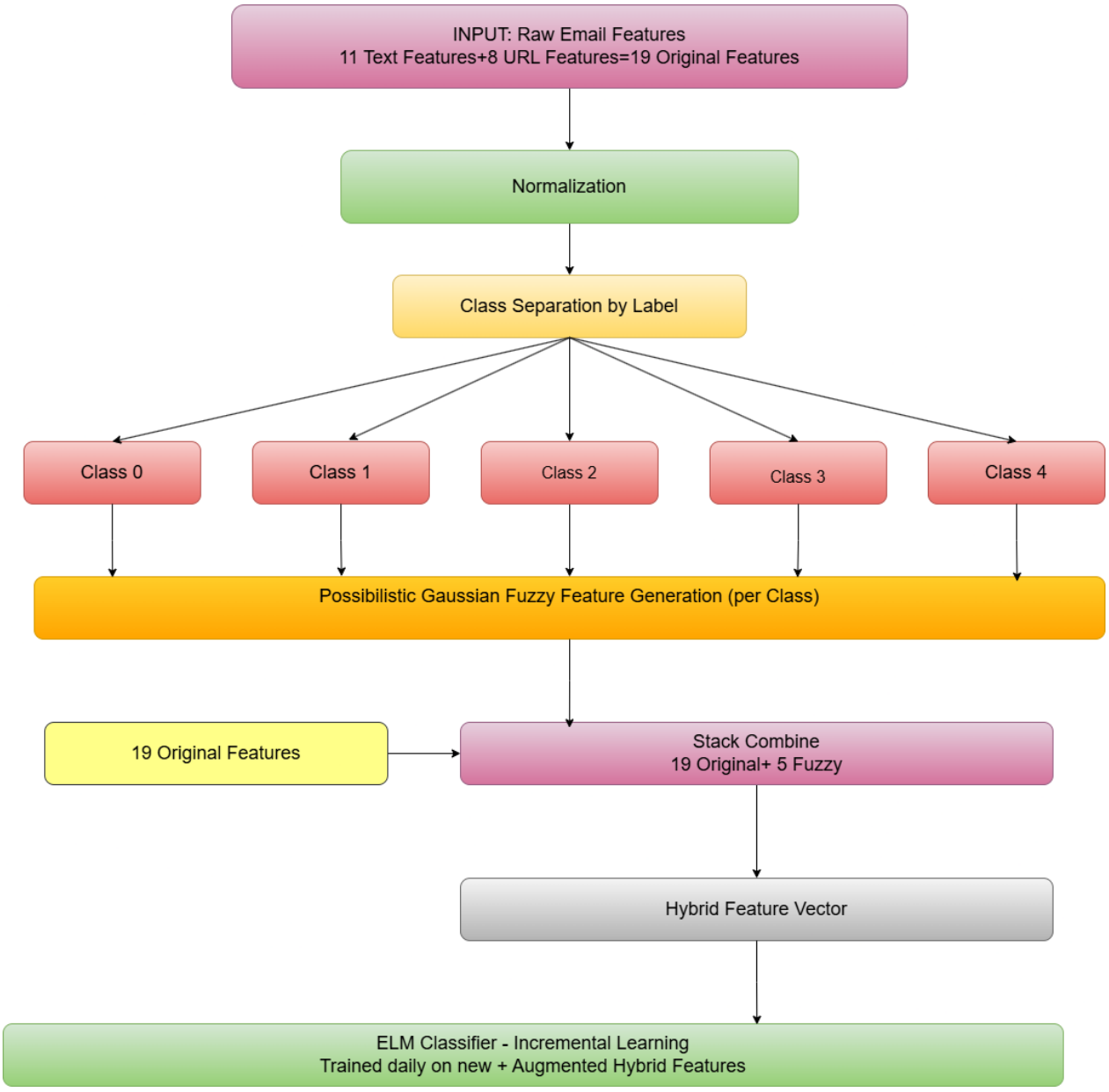


FIGURE 3.2: PGFDG-Based Feature Extraction

Other features are: Subspace Memberships (computed on feature subsets), Homogeneity(relationship with class mean), Outlier Score (computed using

modified Z-score and median absolute deviation), Stability (distance to median), and Fuzzy Entropy.

Among the 16 fuzzy features, five most representative features—Typicality, Density Score, Homogeneity, Outlier Score, and Fuzzy Entropy—are selected and incorporated into the normalized manual features.

A correction is applied to ensure that the fuzzy features are stored in the correct order using boolean masking.

The PGFDG-based fuzzy modeling approach improves robustness by incorporating uncertainty, structural density, and anomalies to improve class separation and generalization capabilities in the incremental social engineering attack detection problem.The detailed procedure for generating the fuzzy descriptors is presented in Algorithm 1.

Algorithm 1 Fuzzy Probability-Guided Feature Descriptor Generator (Fuzzy-PGFDG)

Require: $X \in \mathbb{R}^{n\times 19}$, $y \in \mathbb{R}^n$, $k = 5$, $\varepsilon = 10^{-6}$
Ensure: $H \in \mathbb{R}^{n\times 24}$

$\hat{X} \leftarrow (X - \mu_X)/\sigma_X$
for each class $c \in \{1,\ldots,C\}$ **do**
  $X_c \leftarrow \{\hat{x}_i : y_i = c\}$
  $\mu_c \leftarrow \frac{1}{|X_c|}\sum \hat{x}_i$
  $\Sigma_c \leftarrow \frac{1}{|X_c|-1}\sum(\hat{x}_i - \mu_c)(\hat{x}_i - \mu_c)^T + \varepsilon I$
  $M_c \leftarrow \text{median}(X_c)$
  $\text{MAD}_c \leftarrow \text{median}(|\hat{x}_i - M_c|)$
  $r_c \leftarrow \text{percentile}_{25}(\|\hat{x}_i - \mu_c\|_2)$
end for
for each sample $\hat{x}_i$ **do**
  $d_M \leftarrow \sqrt{(\hat{x}_i - \mu_c)^T \Sigma_c^{-1}(\hat{x}_i - \mu_c)}$
  $T_c \leftarrow \exp\left(-\frac{1}{2}d_M^2\right)$
  $D_c \leftarrow \frac{|\{x_j \in X_c : \|x_j - \hat{x}_i\|_2 \le r_c\}|}{|X_c|}$
  $\mathcal{H}_c \leftarrow \frac{\hat{x}_i \cdot \mu_c}{\|\hat{x}_i\|_2 \|\mu_c\|_2}$
  $O_c \leftarrow \min\left(1.0, \frac{0.6745\|\hat{x}_i - M_c\|_2}{\|\text{MAD}_c\|_2 + \varepsilon}\right)$
  $t \leftarrow T_c$
  $\bar{t} \leftarrow 1 - t$
  $E_c \leftarrow -t\log_2(t+\varepsilon) - \bar{t}\log_2(\bar{t}+\varepsilon)$
  $\phi_i \leftarrow [T_{y_i}, D_{y_i}, \mathcal{H}_{y_i}, O_{y_i}, E_{y_i}]$
end for
$F \leftarrow [\phi_1 | \ldots | \phi_n]^T$
Select top k least-correlated fuzzy features
$H \leftarrow [\hat{X} \mid F_S]$
return H

3.3.6 Data Augmentation

For the purpose of reducing the effects of class imbalance and catastrophic forgetting in the incremental learning framework, a supervised DAIL (Deterministic Alternating Interleaving Logic) data augmentation approach is proposed. The proposed approach uses the learned clusters of previous tasks to favor past class patterns when new data samples arrive.

As shown in 3.3, the augmentation procedure is dependent on the hybrid features derived from the previous training phases. The data points are categorized based on their cluster labels, and for each category, the major class label is determined. The number of data points to be generated for the augmentation procedure is regulated by an augmentation ratio parameter α, which determines the ratio of the augmentation procedure to the number of new data points entering the system. The number of augmentations is limited to enable proper pairing.

The synthetic instances are created through an alternating interleaving mechanism of features. Two parent instances are randomly chosen from the same cluster. The synthetic instances are created through an alternating fashion of choosing feature values from the two parent instances, where even-indexed features are chosen from one parent instance and odd-indexed features are chosen from the other parent instance, and then switched for the second synthetic instance. This deterministic interleaving process ensures that the characteristics of the class are maintained while new combinations of feature patterns are created. Since both parent instances belong to the same cluster, the labels are maintained.

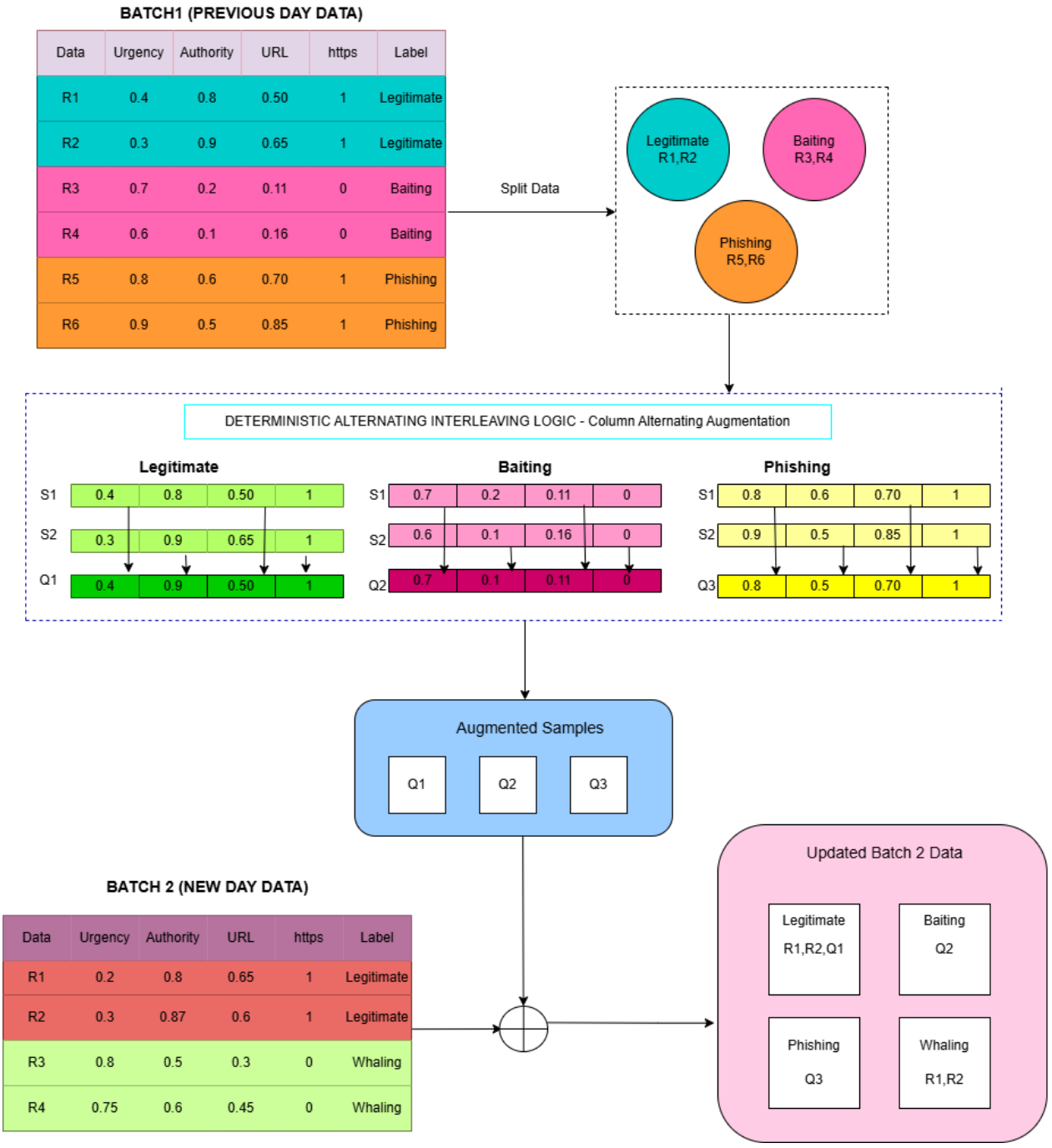


FIGURE 3.3: Data Augmentation- Column Alternating Approach

Each synthesized example is then labeled with the strongest class label from its source cluster, and the cluster labels are kept. The method obtains the expanded feature matrix, class labels, cluster labels, and a summary of the number of synthesized examples per class.

In conclusion, the DAIL method is a simple and supervised augmentation technique for controlled synthetic data generation. The method assists in preserving class distributions from previous learning without storing the whole past data, thus facilitating stable incremental learning and improving model robustness for dynamic social engineering attack detection.The detailed procedure of the DAIL augmentation process is presented in Algorithm 2.

Algorithm 2 DAIL Column-Alternating Augmentation

Require: $S^{(d-1)} \in \mathbb{R}^{m\times 124}$, $y^{(d-1)} \in \mathbb{R}^m$, $\mathrm{CID}^{(d-1)} \in \mathbb{R}^m$, $X_{\text{new}}^{(d)} \in \mathbb{R}^{p\times 124}$, α, K

Ensure: $X_{\text{total}}^{(d)} \in \mathbb{R}^{(p+q)\times 124}$, $y_{\text{total}}^{(d)} \in \mathbb{R}^{p+q}$

Phase 1: Per-Cluster Augmentation Budget

$n_{\text{budget}} \leftarrow \alpha \cdot p$

for $k = 1$ to K **do**

$S_k \leftarrow \{s_i \in S^{(d-1)} : \mathrm{CID}_i = k\}$

$n_k \leftarrow |S_k|$

$n_k^{\text{aug}} \leftarrow \min\left(\left\lfloor \frac{n_k}{2} \right\rfloor, \left\lfloor \frac{n_{\text{budget}}}{K} \right\rfloor\right)$

$n_k^{\text{aug}} \leftarrow n_k^{\text{aug}} - (n_k^{\text{aug}} \bmod 2)$

end for

Phase 2: Column-Alternating Generation

$X_{\text{aug}} \leftarrow \emptyset$, $y_{\text{aug}} \leftarrow \emptyset$

for $k = 1$ to K **do**

$\{s_1, s_2, \ldots, s_{n_k^{\text{aug}}}\} \leftarrow$ random samples from S_k (without replacement)

for $i = 1, 3, 5, \ldots, n_k^{\text{aug}} - 1$ **do**

for $j = 0$ to 123 **do**

$$Q_1[j] = \begin{cases} s_i[j] & \text{if } j \bmod 2 = 0 \\ s_{i+1}[j] & \text{if } j \bmod 2 = 1 \end{cases}$$

$$Q_2[j] = \begin{cases} s_{i+1}[j] & \text{if } j \bmod 2 = 0 \\ s_i[j] & \text{if } j \bmod 2 = 1 \end{cases}$$

end for

$y_{Q_1} = y_{Q_2} \leftarrow \mathrm{mode}(\{y_i : \mathrm{CID}_i = k\})$

$X_{\text{aug}} \leftarrow X_{\text{aug}} \cup \{Q_1, Q_2\}$

$y_{\text{aug}} \leftarrow y_{\text{aug}} \cup \{y_{Q_1}, y_{Q_2}\}$

end for

3.3.7 ELM Classifier

The Extreme Learning Machine (ELM) classifier provides an extremely efficient method for training a neural network classifier compared to the standard training method. Unlike the MLP classifier, which needs to be trained using the backpropagation method, the ELM classifier initializes the weights and biases of the hidden layers randomly and keeps them fixed throughout the entire training process. This reduces the computational overhead involved in training a neural network classifier. Once the transformation of the hidden layers is determined, the ELM classifier computes the output weights in one step, allowing for extremely fast training.

Given an input matrix $X \in \mathbb{R}^{N\times 150}$ and the corresponding output label matrix $Y \in \mathbb{R}^{N\times K}$, the hidden layer output of ELM is computed as

$$H = \sigma(\mathbf{1}\beta_0 + XW) \tag{3.6}$$

where $W \in \mathbb{R}^{150\times L}$ is a randomly initialized weight matrix, $\beta_0 \in \mathbb{R}^{1\times L}$ is a random bias row vector, L is the number of hidden neurons (here $L = 150$), and $\mathbf{1} \in \mathbb{R}^{N\times 1}$ is a column vector of ones used to broadcast the bias across N rows. The activation σ is the sigmoid function:

$$\sigma(x) = \frac{1}{1+e^{-x}} \tag{3.7}$$

This activation function projects the actual inputs to the range $(0,1)$, which helps the hidden layer to produce a smooth non-linear projection of the fused features.

Since W and β_0 are random variables and are fixed, the hidden layer can be treated as a feature projector that projects the input into an informative representation very quickly.

Once the hidden layer output matrix $H \in \mathbb{R}^{N \times L}$ is obtained, the output weights $\beta \in \mathbb{R}^{L \times K}$ are computed analytically using the Moore–Penrose pseudoinverse:

$$\beta = H^{\dagger} Y \tag{3.8}$$

with one common expression for the pseudoinverse (when $H^T H$ is invertible) given by

$$H^{\dagger} = (H^T H)^{-1} H^T \tag{3.9}$$

This provides the least squares solution that directly relates the features of the hidden layers to the desired outputs in a single linear solve. However, in practice, to improve numerical stability, especially when $H^T H$ is close to being singular, a regularized solution is usually used:

$$\beta = \left(H^T H + \lambda I\right)^{-1} H^T Y \tag{3.10}$$

where $\lambda > 0$ is a small regularization parameter and I is the $L \times L$ identity matrix.

With the application of this analytical training method, the ELM classifier is able to efficiently learn the decision boundaries based on the combined latent features obtained from the Stacked Autoencoders. This is particularly useful in scenarios

that require rapid model training or system update. The complementary nature of the ELM classifier with the MLP classifier further enhances the robustness of the system, which is capable of rapid and precise identification of various patterns of social engineering attacks.

As shown in Algorithm 3, the Extreme Learning Machine computes the hidden layer outputs and the output weights analytically.

Algorithm 3 Algorithm for Extreme Learning Machine (ELM)

Input: Training feature matrix X, label vector Y, number of hidden neurons N_h, activation function
Randomly initialize hidden layer weights W_h and biases b_h
Compute hidden layer output matrix H:
for each sample x_i in X **do**
    $h_i = activation(W_h \cdot x_i + b_h)$
end for
Compute output weights β using Moore–Penrose pseudoinverse:
$\beta = pinv(H) \cdot Y$
Training complete: ELM model is ready after obtaining β
Prediction: For a new sample x_{new}:
$h_{\text{new}} = activation(W_h \cdot x_{\text{new}} + b_h)$
Output label prediction: $y_{\text{new}} = h_{\text{new}} \cdot \beta$

3.3.8 Incremental Learning

The proposed system uses a structured four-day incremental learning method to simulate the occurrence of social engineering attack types in the real world. Instead

of training a single model, the classifier is updated as new classes and samples are added. This method enables the system to adapt continuously without catastrophic forgetting.

On Day 1, the training of the model is performed on the first three classes: Legitimate, Phishing, and Baiting. A fixed number of samples for each class are selected to form the initial training set. The hybrid feature extraction is performed by the stacked autoencoder (Pipeline 1) and fuzzy PGFDG features (Pipeline 2), resulting in a 124-dimensional feature space. The dataset is split using an 80/20 stratified train-test split, and an Extreme Learning Machine (ELM) classifier is trained. The number of hidden neurons is dynamically adjusted as $200 \times n_{\text{classes}}$, which is adjusted based on the number of classes.

On Day 2, new samples are added from selected classes. Instead of retraining the model from scratch, the approach includes supervised DAIL augmentation using the stored Day 1 features. New samples are created cluster-wise and combined with the new incoming samples. The pool of samples is then divided and used to train a new ELM model. After the successful training of the model, the old model instance is deleted to save memory.

On Day 3, a new class (Pretexting) is introduced, thus adding to the number of classes in the classification task. Once again, the same procedure is followed: new data is passed through frozen encoders from Day 1 to maintain consistency in the feature space, DAIL augmentation is performed based on the representations from Day 2, and a new ELM is learned. The size of the hidden layer is automatically determined based on the new number of classes.

On Day 4, all five classes (Legitimate, Phishing, Baiting, Whaling, and Pretexting) are trained together. New data is combined with the learned features from Day 3, and a new ELM is trained. Confusion matrix and test accuracy are computed to check the performance. In all stages, the feature encoders are frozen after Day 1 to ensure a fixed feature space.

This incremental learning approach has several advantages. Firstly, it is possible to incrementally add new attack types without the need to retrain on the whole past data. Secondly, the supervised DAIL learning approach assists in reinforcing past class distributions learned by the model, hence combating catastrophic forgetting. Thirdly, the dynamic scaling of hidden neurons assists the ELM classifier in adapting to the increasing complexity of the classes. Finally, past model copies are deleted after each update, hence promoting efficient memory use.

The incremental learning approach hence assists in adaptive, scalable, and memory-efficient social engineering attack detection.

CHAPTER 4

EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS

4.1 DATASET DESCRIPTION

In order to avoid the classifier being biased towards the majority classes, it includes a well-balanced and multi-modal dataset with 2,500 samples that are evenly distributed among the five categories of social engineering attacks, with around 500 samples each category, as demonstrated in in 4.1.

TABLE 4.1: Dataset Description

Attack Category	**Number of Samples**	**Description**
Phishing	496	Fraudulent messages aiming to steal sensitive information.
Whaling	500	High-profile targeting attacks aimed at executives.
Pretexting	518	Social manipulation using fabricated scenarios to extract data.
Baiting	485	Luring users using fake offers or malicious links.
Legitimate	501	Non-malicious samples used as control/normal data.
Total	**2500**	Balanced dataset across all five classes.

4.1.1 Dataset Composition

- Phishing Attacks: 496 examples of widespread, fraudulent emails that target private information and login passwords.
- Whaling Attacks: 500 examples of targeted emails sent to high-value employees and executives.

- Pretexting Attacks: 518 instances of dishonest correspondence creating fictitious connections or authority.
- Baiting Attacks: 485 samples contain enticing communications with malicious attachments or links.
- Legitimate Communications: 501 examples of legitimate communications and real organizational data samples.

4.1.2 Dataset Distribution

Baiting, legitimate, phishing, pretexting, and whaling are the five categories of social engineering attack types that make up the dataset's 2,500 labeled email examples. Each category has between 485 and 518 examples, guaranteeing a nearly balanced class distribution that prevents any one class from overshadowing the learning process. The dataset is separated into four days of incremental learning, in contrast to the usual static dividing of the train and test sets. This simulates a realistic real-world streaming scenario in which fresh data and attack classes are gradually introduced over time.

An 80:20 stratified split is applied separately to the whole pool of examples for each incremental learning day, which consists of both fresh real instances and DAIL-generated synthetic examples from the cluster storage from the day before. A bias toward majority classes in the assessment process is prevented by the stratified split, which guarantees that the class distribution in both the training and test sets appropriately reflects the distribution of active courses for that specific day. The daily distribution is as follows:

TABLE 4.2: Dataset Distribution

Day	Classes Active	New Samples	Augmented	Total Pool	Train (80%)	Test (20%)
Day 1(baiting,phishing,legitimate)	3	700	0	700	560	140
Day 2(baiting,phishing,legitimate,whaling)	4	450	180	630	504	126
Day 3(baiting,phishing,legitimate,whaling,pretexting)	5	576	220	796	636	160
Day 4(baiting,phishing,legitimate,whaling,pretexting)	5	775	348	1,123	898	225
Total	–	**2,501**	**748**	**3,249**	**2,598**	**651**

The per-class sample composition on each day mentioned in 4.2. represents three different scenarios that the incremental system needs to deal with. In the first scenario, a class receives new real samples and new samples from previous days, which gives a strong training example. In the second scenario, a class receives no new real samples but receives DAIL-augmented samples, which is the most important scenario for preventing catastrophic forgetting, which happens on Day 2 for Baiting and Phishing and on Day 3 for Legitimate and Whaling. In the third scenario, a new class emerges for the first time with only real samples and no augmentation history, which happens on Day 2 for Whaling and on Day 3 for Pretexting.

4.1.3 Feature Representation

Text Modality:Email body and subject line content transformed into 300-dimensional vectors by using TF-IDF weighting and subsequent matrix factorization.

URL Modality:URLs are extracted and represented as 68-dimensional feature vectors that encode syntactic characteristics, domain information, and structural patterns.

4.2 ECOSYSTEM

4.2.1 Hardware Configuration

Google Colab's free-tier runtime provides the following hardware configuration, which was used throughout model development:

- **RAM:** 16 GB System memory
- **Storage:**Google Drive integration for accessing datasets and saving model checkpoints.
- **Computational Device:** CPU-based processing.
- **Execution Environment:**Cloud-based runtime with automatic session management

4.2.2 Software Environment

The entire system was developed using Python in the Google Colab environment, supplemented with essential machine learning libraries. The main software components used :

- **Programming Language:** Python 3.8.10
- **Machine Learning Libraries:** scikit-learn 1.1.3, NumPy 1.23.5, Pandas 1.5.3
- **Data Processing:** Scikit-learn preprocessing and feature scaling utilities.

- **Development Environment:** Jupyter Notebook on Google Colab (cloud-based, CPU instances)

4.3 EXPERIMENTS CONDUCTED

4.3.1 Incremental Learning Scenario Design

The data was divided into four sequential pieces, each representing four business days, in order to more closely resemble a real-world example of incremental learning, when social engineering attacks change over time and new attack classes appear gradually. A fresh collection of examples is provided every day, and starting on Day 3, new attack classes that weren't there in the days before start to appear. This is done specifically to assess the model's capacity to absorb new information while maintaining the knowledge acquired from earlier days—the stability-plasticity issue that the DAIL method resolves.

Day 1 — Initial Training (3 Classes)
Initially, 700 samples from three attack classes—250 legitimate, 250 phishing, and 200 baiting attacks—were used to set up the system. On this day, all of the TF-IDF, SVD, and scaler transformers are applied, and the Stacked Autoencoder is trained from scratch to create the foundation for the feature space. For the duration of the incremental learning period, the frozen transformers from Day 1 will be used on all subsequent days to keep the feature space constant. On Day 1, no augmentation is carried out because there is no prior history to draw from.

Day 2 — First Incremental Update (4 Classes)

The second batch introduces 450 new samples — 150 Legitimate and 300 Whaling. Notably, Phishing has zero new real arrivals on Day 2, creating the risk that the updated model will forget previously learned Phishing patterns. DAIL augmentation is applied with α, generating some synthetic Phishing samples from Day 1's cluster,few augmented Baiting samples, and few augmented Legitimate samples — totalling augmented samples. These are merged with the new samples to form a sample total training pool.

Day 3 — Second Incremental Update (5 Classes)

Test accuracy dropped to 86.25% following the introduction of Pretexting. Pretexting shares formal, authority-driven language features with both Legitimate and Whaling, causing inter-class confusion. Legitimate had zero new arrivals but was sustained through DAIL-augmented samples, demonstrating effective catastrophic forgetting prevention.

Day4 -Third Incremental Update (5 Class)

Test accuracy reached 83.56% on 225 test samples with all five classes now active. The Day 4 total training pool comprised 1,123 samples — 775 new real samples combined with 348 DAIL-augmented samples drawn from the Day 3 cluster storage. This is the largest training pool across all four days, reflecting the cumulative growth of the incremental system.The day wise data configuration is clearly mentioned in 4.3.

TABLE 4.3: Day wise data configuration

Day	New Samples	Classes Active	New Class Introduced	α	Augmented Samples	Total Pool	Train (80%)	Test (20%)
Day 1(baiting,phishing,legitimate)	700	3	—	—	0	700	560	140
Day 2(baiting,phishing,legitimate,Whaling)	450	4	Whaling	0.40	180	630	504	126
Day 3(baiting,phishing,legitimate,Whaling,pretexting)	575	4	Pretexting	0.50	220	796	636	160
Day 4(baiting,phishing,legitimate,whailing,pretexting)	775	5	None	0.60	348	1123	898	225

4.3.2 Stacked Autoencoder Experiment

The first feature extraction was developed to produce deep and dense representations from the raw email text and URL features using a two-step approach: dimensionality reduction and autoencoder-based representation learning.

Text Feature Extraction

The body and subject of each email were concatenated and preprocessed using lemmatization and regex-based noise removal. A TF-IDF vectorizer with max vocabulary of 2,000 features and unigram-bigram range (1,2) was used to convert the preprocessed text into a sparse matrix. Truncated SVD with 200 components was used to reduce the sparse matrix to a dense 200-dimensional representation. A StandardScaler was used to normalize the output of SVD before passing it to the autoencoder.

The text autoencoder was designed with an architecture of 200→100→50→100→100→200, employing *Leaky ReLU* activation functions for all hidden layers and a linear activation for the output layer. The bottleneck layer provides a 50-dimensional encoded representation of the text. The model was trained using the Adam optimizer with Mean Squared Error (MSE) loss for 300 epochs and a batch size of 64.

URL Feature Extraction

Four URL characteristics — URL length, number of dots, number of slashes, and presence of HTTPS — were manually feature-engineered. In addition, a character-level TF–IDF vectorizer (`analyzer='char_wb', ngram_range=(3,5), max_features=512`) was applied to the raw URL strings, followed by Truncated

SVD with 64 components. These features were horizontally stacked to form a 68-dimensional URL feature matrix, which was normalized before training the autoencoder.

The URL autoencoder architecture was 68→60→50→60→68, again using *LeakyReLU* activation functions, yielding a 50-dimensional encoded URL representation. The model was trained using the same configuration for 300 epochs.

Frozen Feature Space

One of the most important design choices is that the autoencoders and all the transformers (TF-IDF, SVD, and scalers) are trained only on Day 1 data, and their weights are set to be frozen for Days 2, 3, and 4. This guarantees that all 124-dimensional feature vectors for all four days are in the same feature space, allowing for a fair comparison and classification across all four days.

The two 50-dimensional outputs are concatenated to create the 100-dimensional output vector

4.3.3 Fuzzy-PGDFG Experiment

The second extracts interpretable, uncertainty-aware features using a fuzzy logic framework applied to manually engineered content features.

Manual Feature Engineering

Eleven text features are extracted per sample — urgency score, authority score, financial keyword score, credential keyword score, secrecy score, action score, routine score, length score, formality score, time pressure score, and grammar

score. Eight URL features are extracted — URL presence, URL length ratio, dot count ratio, digit ratio, HTTPS flag, Shannon entropy, special character ratio, and subdomain count ratio. All 19 features are normalized using StandardScaler.

Fuzzy Feature Generation

For each class cluster, the Fuzzy-PGFDG computes 16 fuzzy descriptors per sample using Mahalanobis distance, neighborhood density estimation, subspace membership analysis, and entropy-based uncertainty quantification. From these 16, five diverse and non-redundant features are selected:

- **Typicality:** Computed as $\exp(-0.5 \times d_M^2)$ where d_M is the Mahalanobis distance — measures how representative the sample is of its class centroid.
- **Density:** Local cluster compactness is encoded by the ratio of class neighbors within the 25th percentile radius.
- **Homogeneity:** Intra-class homogeneity is encoded by the Pearson correlation coefficient between the sample descriptor and class mean.
- **Outlier Score:** a modified Z-score that uses the median absolute deviation to identify samples that are close to class borders.
- **Fuzzy Entropy:** This measure encodes assignment uncertainty and is the normalized Shannon entropy of the typical/atypical membership pair.

2's final output, a 24-dimensional hybrid vector (19 manual + 5 fuzzy), is concatenated with 1's 100-dimensional output to create a 124-dimensional composite feature vector for classification.

Data Augmentation Experiment

The DAIL column-alternating augmentation was applied at the start of Days 2, 3, and 4 using the previous day's stored combined 124-dimensional feature matrix as the source pool. For each class cluster, pairs of samples are randomly selected without replacement, and two synthetic samples are generated per pair by alternating feature column values between the two parent samples:

$$Q_1[j] = \begin{cases} S_1[j] & \text{if } j \bmod 2 = 0 \\ S_2[j] & \text{if } j \bmod 2 = 1 \end{cases} \qquad Q_2[j] = \begin{cases} S_2[j] & \text{if } j \bmod 2 = 0 \\ S_1[j] & \text{if } j \bmod 2 = 1 \end{cases}$$

This guarantees that all generated samples remain within their parent cluster's feature distribution.The per-class augmentation outcomes across the three augmented days are mentioned in Table 4.4

TABLE 4.4: Per-class augmentation outcomes

Day	Class	Prev. Cluster Size	Samples Generated
Day 2	Baiting	200	60
Day 2	Legitimate	250	60
Day 2	Phishing	250	60
Day 2	Whaling	0	New class
Day 3	Baiting	300	60
Day 3	Legitimate	210	80
Day 3	Phishing	60	60
Day 3	Whaling	300	80
Day 3	Pretexting	0	New class
Day 4	Baiting	180	90
Day 4	Legitimate	90	40
Day 4	Phishing	156	78
Day 4	Whaling	90	40
Day 4	Pretexting	300	100

4.3.4 Classifier Training Experiment - ELM

ELM Training

For each day, the ELM was trained on the standardized combined feature matrix. A grid search was performed over dynamically computed n_{hidden} candidates $\left(\frac{n_{\text{train}}}{6}, \frac{n_{\text{train}}}{4}, \frac{n_{\text{train}}}{3}\right)$ and three activation functions (*sigmoid*, *ReLU*, *tanh*). Output weights were computed analytically using the Moore–Penrose pseudoinverse with L_2 regularization ($\lambda = 10^{-2}$). The best configuration was selected using the gap-penalized scoring criterion.

4.4 PERFORMANCE ANALYSIS

4.4.1 SAE - ELM Analysis

First evaluates deep latent feature representations learned using TF-IDF, truncated SVD, and a Stacked Autoencoder (SAE) before ELM classification. With 200 hidden neurons and ReLU activation, the best combination demonstrated 84.67% accuracy, indicating that encoded representations with lower dimensionality can successfully capture semantic and structural patterns in text and URL feature data. Whereas excessively large hidden layers resulted in reduced generalization, performance was stable for moderate hidden sizes (100–300 neurons). The comparatively poorer performance on related attack classes suggests that deep feature representations might not be enough to capture uncertainty and class overlap, even clearly has robust representation learning capabilities.

TABLE 4.5: Performance Comparison of ELM Configurations 1

Hidden Neurons	Activation Function	Accuracy	Macro F1-Score
100	Sigmoid	0.8493	0.85
150	Sigmoid	0.8573	0.86
200	ReLU	0.8467	0.84
300	ReLU	0.8387	0.84

With 150 hidden neurons and Sigmoid activation, the best combination was found, with 95.60% accuracy and a macro F1-score of 0.9601 as mentioned in 4.5. This clearly demonstrated a huge improvement over first due to the usage of fuzzy features.

4.4.2 Original Features+Fuzzy Features - ELM Analysis

For behavioral features that are manually created and enhanced using fuzzy membership descriptors, shows itself to be a reliable classifier. The concatenated model that combined fuzzy attributes significantly increased the baseline model's accuracy to 95.60% from 92.40%. Particularly in semantically related assault types like phishing and whaling, the addition of Typicality, Density, Homogeneity, Outlier Score, and Fuzzy Entropy significantly aided in class separation. The ELM classifier's hyperparameter optimization showed that a modest number of hidden neurons (100–300) with sigmoid or ReLU activation functions performed best. The experiment unequivocally demonstrates that using fuzzy modeling to manage uncertainty enhances generalization and robustness in the detection of social engineering attacks.

As mentioned in 4.6 The best-performing configuration was achieved with 150 hidden neurons using Sigmoid activation, yielding 95.60% accuracy and a macro

TABLE 4.6: Performance Comparison of ELM Configurations 2(Hybrid Model)

Hidden Neurons	Activation Function	Accuracy	Macro F1-Score
100	Sigmoid	0.940	0.9225
150	Sigmoid	0.9450	0.9401
200	ReLU	0.9560	0.9563
300	ReLU	0.9340	0.9242

F1-score of 0.95, demonstrating a substantial improvement over 1 due to the integration of fuzzy features.

4.4.3 Day-wise Incremental Performance - ELM

The proposed system was evaluated across four incremental learning days using the Extreme Learning Machine (ELM) as the primary classifier. The ELM was trained on the 124-dimensional combined feature vector produced by the dual-pipeline feature extraction framework — 100 dimensions from the Stacked Autoencoder and 24 dimensions from the Fuzzy-PGFDG. For each day, a gap-penalized hyperparameter search was conducted to identify the optimal hidden layer size and activation function. The complete day-wise performance is summarized below in table 4.7

4.4.4 ELM Hyperparameter Selection Analysis

For each incremental day, the ELM hyperparameter search evaluated all combinations of dynamically computed n_{hidden} candidates and three activation functions (sigmoid, relu, and tanh) using the gap-penalized scoring criterion.

TABLE 4.7: Day-wise Incremental Analysis

Batch (Day)	**Training Samples**	**Test Samples**	**ELM Train Accuracy (%)**	**ELM Test Accuracy (%)**	**Train-Test Gap (%)**
Day 1 (baiting, phishing, legitimate)	560	140	99.64	92.86	6.78
Day 2 (baiting, phishing, legitimate, whaling)	453	114	99.60	88.89	10.71
Day 3 (baiting, phishing, legitimate, whaling, pretexting)	609	153	97.80	86.25	11.55
Day 4 (baiting, phishing, legitimate, whaling, pretexting)	898	225	96.55	83.56	12.99

$$\text{Score} = \text{Test Accuracy} - 0.5 \times \max(0, \text{Gap} - 0.10) \tag{4.1}$$

The selection of topologies that strike a compromise between prediction accuracy and generalization capability is encouraged by this criterion, which penalizes any train-test gap more than 10% by lowering the effective score.

TABLE 4.8: Gap-Penalized Hyperparameter Selection Across Incremental Days

Day	n_{hidden}	**Best** n_{hidden}	**Act.**	**Train (%)**	**Test (%)**	**Gap (%)**	**Score**
Day 1(baiting,phishing,legitimate)	93,140,186	186	tanh	99.64	92.86	6.78	0.9286
Day 2(baiting,phishing,legitimate,whaling)	75,113,151	126	relu	99.60	88.89	10.71	0.8854
Day 3(baiting,phishing,legitimate,whaling,pretexting)	101,152,203	159	sigmoid	97.80	86.25	11.55	0.8548
Day 4(baiting,phishing,legitimate,whaling,pretexting)	125,188,251	299	relu	96.55	83.56	12.99	0.8206

Over the course of the four days,as mentioned in Table 4.8 it was observed that while larger values of n_{hidden} result in training data memorization, lesser values lead to better generalization but worse training accuracy. Instead of selecting the largest candidates, the gap-penalized selector selects intermediate settings that offer the optimal balance between generalization and accuracy. When more attack classes are introduced, the non-linearity in decision boundaries increases, as evidenced by the best activation function shifting from tanh on Day 1 to relu on Days 2 and 4.

4.4.5 Complete Tabulation of Results

The whole day-by-day outcomes of the incremental learning process are summarized in the master table below, along with comprehensive class data as depicted in table 4.9:

TABLE 4.9: Day-wise Training Distribution and Incremental Performance

Day	Training Set (Class : Samples)	Test Set (Samples)	Training Acc (%)	Testing Acc (%)
Day 1	Baiting: 160, Legitimate: 200, Phishing: 200 (Total: 560)	140	99.64	92.86
Day 2	Baiting: 48, Legitimate: 168, Phishing: 48, Whaling: 240 (Total: 504)	126	99.60	88.89
Day 3	Baiting: 144, Legitimate: 64, Phishing: 124, Pretexting: 240, Whaling: 64 (Total: 636)	160	97.80	86.25
Day 4	Baiting: 180, Legitimate: 112, Phishing: 159, Pretexting: 254, Whaling: 192 (Total: 898)	225	96.55	83.56

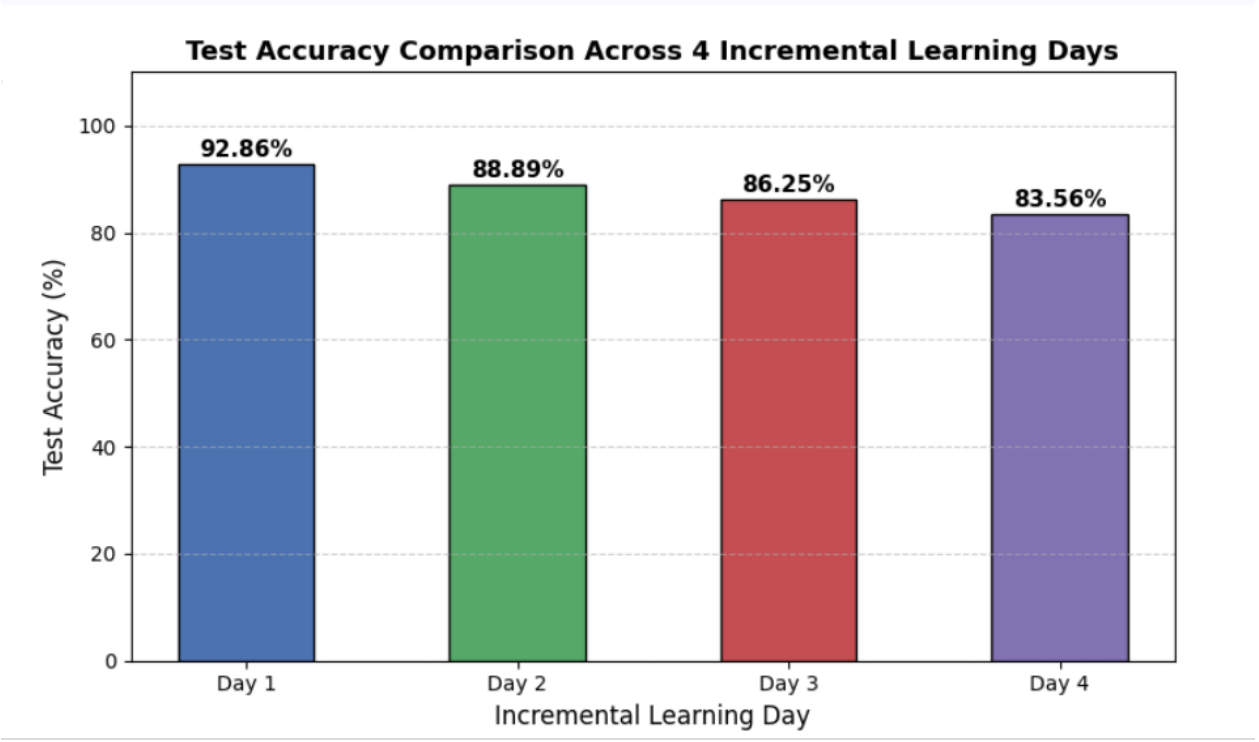


FIGURE 4.1: Test Accuracy Comparison Across Four Incremental Learning Days

With an increasingly larger dataset that increased from 700 to 1,123 samples over the course of the incremental learning process, the ELM's average test accuracy on all four incremental days was 87.89%, correctly classifying all five categories of social engineering attacks: baiting, legitimate, phishing, pretexting, and whaling. DAIL augmentation was also found to be a successful method for information retention in the system, as no previously acquired class was completely forgotten during the incremental learning process.

CHAPTER 5

SOCIAL IMPACT AND SUSTAINABILITY

5.1 SOCIAL IMPACT

The proposed system has a great social impact as it shields individual users from social engineering attacks that target human trust, urgency, and authority instead of technical vulnerabilities. Such attacks include phishing, baiting, pretexting, whaling, and malicious URL attacks, which often lead to financial loss, identity theft, and unauthorized access to accounts. Many of these attacks go unnoticed until a great deal of damage has been done. The system will act as a smart first line of defense that runs continuously in the background, making it unnecessary for users to have advanced knowledge of cybersecurity. This is particularly important for senior citizens, students, and first-time internet users who are more prone to deceptive digital communication.

In addition to the personal security feature, the system enhances the overall cybersecurity of organizations and fosters digital trust. Social engineering attacks against executives and employees can result in fraudulent transactions, breaches of data, and reputation damage. With strong detection performance on all five attack classes, including strong recall on Whaling and strong F1-score on Pretexting in the incremental learning settings, the system assists in early threat detection, lowering the cost of response, and enhancing regulatory compliance. Furthermore, by leveraging open-source technologies and running efficiently on commodity hardware, the system assists in the democratization of cybersecurity,

allowing small businesses, institutions of learning, and NGOs to benefit from cutting-edge cybersecurity protection.

5.2 SUSTAINABILITY

The proposed system is computationally efficient and sustainable. The Extreme Learning Machine calculates the output weights analytically without the need for iterative gradient descent computation, thus eliminating the need for GPU-intensive training. The Stacked Autoencoder is trained only once on the initial data and is left frozen for the rest of the incremental updates, thus eliminating the need for repeated deep learning training cycles. Furthermore, the Fuzzy-PGFDG feature calculations are performed through efficient matrix calculations that can be performed on standard CPU hardware. This is a significant move towards energy efficiency compared to the transformer model or the large language model approach.

Sustainability is further improved by the use of incremental adaptability and memory-efficient design. Unlike conventional machine learning systems, which have to be completely retrained when new types of attacks are discovered, the proposed incremental learning approach incorporates new knowledge without forgetting previously gained knowledge. This increases the useful life of the system. The memory cleanup protocol ensures that old feature matrices and models are deleted after each incremental learning update, regardless of the age of the system. This ensures that the system is scalable without increasing hardware costs, thus improving economic and environmental sustainability.

CHAPTER 6

CONCLUSION AND FUTURE WORK

In conclusion, the proposed incremental learning approach provides an adaptive and scalable method for detecting social engineering attacks. The system integrates stacked autoencoder-based deep feature extraction with PGFDG fuzzy modeling to capture structural patterns, uncertainty, and anomalies. Additionally, the combination of normalized manual features with fuzzy features results in a robust hybrid feature representation that enhances class separability, particularly in scenarios involving overlapping attack classes.

To overcome catastrophic forgetting, where a model loses previously learned information when trained on new data, the supervised DAIL data augmentation process helps reinforce previously learned class knowledge. The proposed system evolves over four stages, increasing from three to five attack classes. The consistency of the feature space is ensured by frozen encoders, which are neural network components that do not update during training. The Extreme Learning Machine (ELM), a type of single-layer feedforward neural network known for rapid training and efficient computation, makes the system suitable for near real-time applications. The proposed system effectively integrates deep learning, fuzzy modeling, data augmentation, and incremental classification.

As future work, adding more new attack types, such as spear phishing and AI scams, can enhance the accuracy. Furthermore, adopting more advanced hybrid genetic approaches for data augmentation can improve learning for the minority class.

REFERENCES

1. N. Altwaijry *et al.*, "Advancing phishing email detection: A comparative study of deep learning techniques," *PLoS One*, vol. 19, no. 3, p. e0299216, Mar. 2024.

2. N. Alsufyani and A. Alzahrani, "Social engineering attack detection using machine learning," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 2,2022.

3. S. Ariyadasa, S. Fernando, and S. Fernando, "Detecting phishing attacks using a combined model of LSTM and CNN," *Int. J. of Advanced and Applied Sciences*, vol. 7, no. 7, pp. 56–67, 2020.

4. S. Bagui, D. Nandi, S. Bagui, and R. J. White, "Machine learning and deep learning for phishing email classification using one-hot encoding," *Journal of Computer Science*, vol. 17, pp. 610–623, 2021, doi:`https://doi.org/~10.3844/jcssp.2021.610.623`

5. S. Bahadoripour, H. Karimipour, A. N. Jahromi, and A. Islam, "An explainable multi-modal model for advanced cyber-attack detection in industrial control systems," *Internet of Things*, vol. 26, p. 101213, 2024.

6. S. Gupta, S. Jain, V. Awasthi, A. Sabharwal, M. Kumar, A. Gaur, and W. M. Khan, "Detection of phishing URL based on text feature extraction," *International Journal of Food and Nutritional Sciences (IJFANS)*, vol. 11, Special Issue 6, pp. 465, 2022.

7. “Multi-modal fusion for anomaly detection in cybersecurity: Integrating NLP with network traffic data and system logs,” *EasyChair Preprint*, 2024.

8. “Multi-modal data fusion techniques for improved cybersecurity threat detection and prediction,” *Baltic Journal of Modern Learning*, Dec. 2024.

9. S. Jesmithaa and A. P. Eliyas, “Detecting phishing attacks using convolutional neural network and LSTM,” in *Proc. 3rd Int. Conf. on Advance Computing and Innovative Technologies in Engineering (ICACITE)*, Greater Noida, India, 2023, pp. 802–807.

10. M. Lansley, S. Kapetanakis, and N. Polatidis, “SEADer++ v2: Detecting social engineering attacks using natural language processing and machine learning,” in *Proc. 2020 Int. Conf. on Innovations in Intelligent Systems and Applications (INISTA)*, 2020, pp. 1–6.

11. S. Madhusudhanan and S. Jaganathan, “Data augmented incremental learning (DAIL) for unsupervised data,” *IEICE Transactions on Information and Systems*, vol. 105, no. 6, pp. 1185–1197, 2022.

12. A. Mahdieh Zabihimayvan, B. Derek Doran, “Fuzzy rough set feature selection to enhance phishing detection,” *arXiv preprint arXiv:1903.05675*, 2019.

13. A. Namavar Jahromi *et al.*, “A deep multi-modal cyber-attack detection model,” *arXiv preprint arXiv:2304.01440*, 2023.

14. M. K. Prabakaran, P. M. Sundaram, and A. D. Chandrasekar, “An enhanced deep learning-based phishing detection mechanism to effectively identify

malicious URLs using variational autoencoders," *IET Information Security*, vol. 17, pp. 423–440, 2023.

15. A. Prasad and S. Chandra, "PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning," *Computers & Security*, vol. 136, p. 103539, 2024.

16. O. K. Sahingoz, E. Buber, and O. Kugu, "A dual-phase deep learning framework for advanced phishing detection using multiple algorithms," *Scientific Reports*, vol. 15, p. 21986, Jul. 2025.

17. B. Senthilnayaki, K. Venkatalakshmi, and A. Kannan, "Intrusion detection system using fuzzy rough set feature selection and modified KNN classifier," *The International Arab Journal of Information Technology*, vol. 16, no. 4, Jul. 2019.

18. M. S. V. Tyagadurgam *et al.*, "Preventing phishing attacks using advanced deep learning techniques for cyber threat mitigation," *Journal of Data Analysis and Information Processing*, vol. 13, pp. 314–330, 2025.

19. C. Xu, Y. Zhan, Z. Wang, and J. Yang, "Multimodal fusion based few-shot network intrusion detection system," *Scientific Reports*, vol. 15, p. 21986, Jul. 2025.

20. Y. Zhang *et al.*, "Detection and classification of phishing websites using machine learning," *Cybersecurity*, vol. 8, no. 1, pp. 1–18, May 2025.

21. P. Vidyasri and S. Suresh, "FDN-SA: Fuzzy deep neural-stacked autoencoder-based phishing attack detection in social engineering," *Computers & Security*, vol. 148, p. 104188, 2025.